

Performance Notes — Dashboard / Sales Register / Reports

load time

Date investigated: 2026-07-07

Status: Investigated, understood, **decided to do nothing for now** (app is correct, just slower on large ranges + the Dashboard landing page). This document exists so future-me can pick it back up without re-deriving everything.

1. The symptom

The Sales Register (and Dashboard) felt slow to load. Question raised: is it a temporary / environmental issue, or a real problem in the code?

2. How it was measured

Temporary `console.log` timing was added around each screen's data fetch, then removed after diagnosis. To re-instrument in future, add these back:

- **Sales Register** — `src/pages/SalesList.tsx`, in the `useEffect` that calls `fetchSalesByDateRange` :

```
const t0 = performance.now();
fetchSalesByDateRange(rangeStart, rangeEnd).then((data) => {
  console.log(`[sales-load] ${data.length} rows in ${Math.round(performance.now() - t0)}ms (${datePres
  ...
```

- **Dashboard** — `src/pages/Dashboard.tsx`, around the `Promise.all([...])` in `loadData` .
- **Reports** — `src/pages/Reports.tsx`, around the `Promise.all([...])` in `loadSales` .

Then open DevTools → Console and switch date filters / reload.

3. The numbers observed (real data, ~1,250 sales rows in "last month")

Screen	Range	Rows	Time
Sales Register	today	2	240ms / 441ms
Sales Register	yesterday	4	210ms
Sales Register	last 7 days	19	708ms (cold) / then fast
Sales Register	this month	19	234ms
Sales Register	last month	1253	1095ms / 983ms
Sales Register	custom (small)	2	~210ms
Sales Register	custom (large)	1080	834ms
Reports	today	2 (+3 returns)	223ms / 436ms
Reports	this month	19 (+12)	266ms
Reports	last month	1253	1046ms
Dashboard (landing)	8 queries incl. last-month 1253	—	1041ms / 1466ms

4. Findings

a) **Small ranges (2–19 rows): ~200–450ms — this is fine.** ~200ms is the network round-trip floor to Supabase (RTT + query). Occasional spikes (440–700ms) are cold-start jitter. Normal for a remote DB. Not a code problem.

b) **Large ranges (~1,250 rows): ~1s — consistent.** Two causes:

- `fetchSalesByDateRange` uses `select('*')` (all ~15 columns) → large payload.
- >1000 rows triggers the PostgreSQL 1000-row cap, so it does **2 sequential** paged queries (`range(0,999)` then `range(1000,...)`), doubling the round-trip.

c) **Dashboard (~1–1.5s every load) — the real everyday pain.** The **landing page** pulls the full **1,253 "last month" rows just to display one number** (the last-month total), plus yesterday/this-month rows, and sums them in JavaScript. It fires 8 queries in parallel, so total ≈ the slowest one (the 1,253-row pull).

NOT the cause (checked and ruled out):

- **Missing index** — ruled out. `sales` already has `sales_sale_date_idx` on `sale_date` (plus indexes on `invoice_number`, `medicine_name`, `payment_mode`, `customer_id`). Confirmed via `select indexname, indexdef from pg_indexes where tablename = 'sales';`. Date-range queries are already index-backed. **Do not "add a sale_date index" — it exists.**
- **Client rendering** — the lists use progressive rendering (150 rows, `useProgressiveList`), so rendering isn't the bottleneck; the fetch is.

Bottom line: partly environmental (Supabase **free tier** cold-start + ~200ms RTT floor, which is "temp"), partly a real code inefficiency (pulling thousands of full rows to compute sums/counts, most visibly on the Dashboard landing page). The numbers are **correct** — only speed is affected.

5. Options considered (with risk) — for the future

Reminder: current numbers are already correct, so any change must produce **byte-identical** results. Bar = a total is `rows.reduce((s,x)=>s+x.total_amount)`; invoice count is `count(distinct invoice_number)`.

Option	Effect	Risk	Notes
A. Dashboard aggregates — compute yesterday/month/last-month totals with SQL <code>sum()</code> + <code>count(distinct invoice)</code> instead of pulling rows	~1.4s → ~250ms on landing page	Moderate, containable	Needs a small additive DB function. Must match the <code>.gte/.lte</code> filter and use <code>count(distinct)</code> . Leave Today card, credit ledger, invoice/return math untouched.
B. Column projection — <code>select</code> only needed columns instead of <code>*</code>	20–40% smaller payloads on big ranges	Higher hidden risk — advised against	That fetch is shared by Sales Register, Reports, printing, returns, discounts. Omitting one used column (e.g. <code>mrp</code> , <code>batch_number</code>) breaks a feature <i>silently</i> . Skip it.
C. Lazy "Last Month" on Dashboard — don't fetch the 1,253 rows until the card is clicked	Removes the biggest pull from the landing page	Zero calculation risk	Same computation, just deferred. No DB change. Good low-risk middle ground.
D. Supabase Pro (~\$25/mo)	Dedicated compute, no idle pausing / cold-start	None (infra)	Removes the "temp" factor entirely.
E. Region check	Could drop the ~200ms RTT floor to ~50ms	None (infra)	Only if the project region is far from where the pharmacy operates. Changing region = project migration.
F. Do nothing	—	None	App is correct; large ranges + Dashboard are "slow but right." This is the chosen option (2026-07-07).

If/when revisiting, recommended order

1. **C (lazy Last Month)** — zero-risk, kills most of the landing-page lag.

2. **A (aggregates)** — bigger win, do it with a **shadow comparison**: compute both the old JS way and the new SQL way, log/assert they match on real data, cut over only once verified. Add a unit test for sum/count parity.
3. Consider **D/E** (infra) if the ~200ms floor / cold-starts still bother you.
4. **Skip B.**

6. Safety approach for Option A (so a calc regression can't ship)

- Keep the current JS computation running alongside the new aggregate temporarily.
- Log any mismatch between the two on real data; only remove the old path once they never differ.
- Never touch the highest-stakes numbers in the same change (Today card, credit outstanding, invoice/return totals).
- Additive DB function only (read-only) — cannot corrupt data.

7. Related code

- Fetch (paginated, `select('*',)` , 1000-row chunking): `fetchSalesByDateRange` in `src/hooks/useSales.ts`
- Dashboard load (8 parallel queries): `loadData` in `src/pages/Dashboard.tsx`
- Reports load: `loadSales` in `src/pages/Reports.tsx`
- Progressive rendering (150 rows): `src/hooks/useProgressiveList.ts`
- Index check query: `select indexname, indexdef from pg_indexes where tablename = 'sales';`